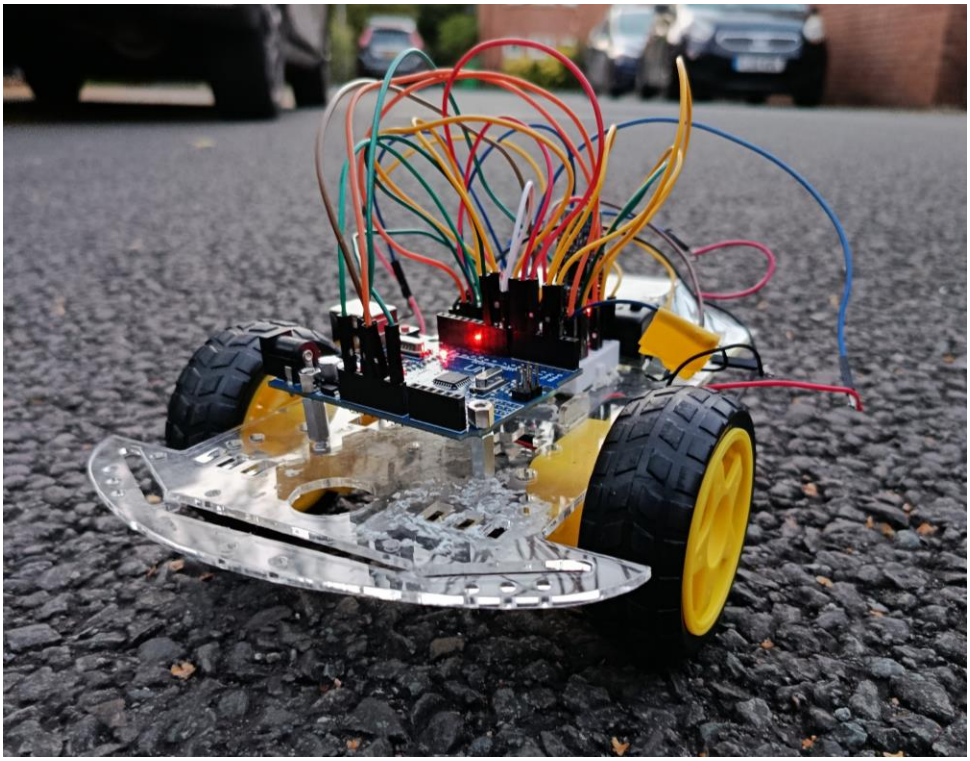
ROBOSOC

RC BUGGY DELIVERY PROJECT



2026-2027

By Sean, Angie and Juza

WORK IN PROGRESS

Contents

Project Overview

- Introduction
- Project Summary
- Parts List
- Acknowledgements

Building The Chassis

- Assembly
- Powering the buggy

Motors

- Introducing the motor driver
- Wiring the motors
- Connecting the motor driver to the Arduino
- Powering the Arduino

Getting the buggy moving

- Enabling the Motor Driver
- Controlling Motor Speed
- Controlling Motor Direction
- Making Functions

Bluetooth Module

- Connecting to your phone
- Communicating with your phone
- Adding functions
- Testing and tweaking

Level 2: Arms

- Intro to level 2
- Creating the arm
- Attaching the servos
- Writing servo functions

FAQs

- Arduino & Code
- Motors & Motor Driver
- Servo Motors
- Bluetooth
- Wiring, Power & Assembly

Introduction

Welcome to the Buggy Project!

We're thrilled to have you with us on this exciting journey into the world of **robotics, problem-solving, and creativity**!

In this project, you will build your own **Arduino-powered remote-control buggy**, inspired by the autonomous delivery robots used in real life to transport food, parcels, and other goods. You'll learn how electronics, programming, and mechanical components come together to create a working robot. And, most importantly, you'll get to build one yourself!

Whether you're completely new to robotics or have some experience already, this guide is designed to support you every step of the way. Our goal is simple: **to spark your curiosity and give you the tools to build a fully functioning buggy your way.** By the end of the project, you'll have a working remote-controlled delivery buggy and a better understanding of the basics of electronics, programming, and robotics.

Throughout the guide, you'll find **step-by-step instructions** covering everything from assembling your buggy and positioning its components, to connecting circuits and programming your Arduino. While the guide contains everything you need to get started, we encourage you to experiment and try things for yourself. Change something, test a different approach, or see what happens when you modify your code. Every tweak is another opportunity to learn!

If you're up for a challenge, try building your buggy yourself before looking at the instructions. You might surprise yourself with what you can figure out! Once you're feeling more confident, you can also take on the **Intermediate Guide**, which provides fewer suggestions and gives you more freedom to design and experiment.

And remember, you're never limited to this guide. There are countless robotics projects, tutorials, and communities online that you can explore. If something doesn't work the first time, don't be discouraged! Troubleshooting and learning from mistakes are a huge part of engineering!

If you're feeling overwhelmed or unsure where to start, don't worry. There's absolutely no shame in following the guide closely, **it's here to make things easier for you**. And if you ever get stuck or have a question, the Buggy Team is always here to help.

So, are you ready to build your own delivery robot?

Let's get building! And remember, this project is about more than just the finished buggy. It's about experimenting, solving problems, and learning along the way.

Project Summary

In this project, you will build your own Arduino-powered remote-control buggy, inspired by the delivery robots used in real life to transport food, parcels, and other goods.



You will learn how to assemble the buggy, connect electronic components, and program an Arduino to control its movement. The project will introduce you to key concepts in electronics, robotics, and programming, while giving you the opportunity to test and customise your own robot.

By the end of the project, you will have a working remote-controlled delivery buggy that can be used to complete a variety of challenges and delivery tasks!

Parts List

- 1x Chassis
- 2x DC motors
- 1x Arduino Uno
- 1x Motor Driver (TB6612FNG)
- 4x Spacers
- 2x Screw Terminals

- 2x Wheels
- 1x HM-10 Bluetooth module
- 1x Arduino cable
- 1x Breadboard
- 2x Velocimetry code wheel
- 6x M3*30 Screws
- 6x M3*8 Screws
- 8x M3 Nuts
- 1x Switch
- 4x Fasteners
- 4x L12 Spacers
- 1x Hammer caster
- 1x Battery box
- [2x Servo Motors]
- [+1 Battery Pack]

Acknowledgements

This project and guide build heavily upon the work, ideas, and contributions of previous Buggy Teams. We would like to acknowledge and thank everyone who has contributed to the development of the project over the years.

A special thanks goes to Alessandro Sica for his significant contributions to the project and for helping to develop and improve the buggy experience for future participants.

This guide is intended as an educational resource and builds upon existing knowledge, designs, and materials developed by previous teams. We do not claim ownership or credit for work that originated from others, and we are grateful to those who have contributed to the project before us.

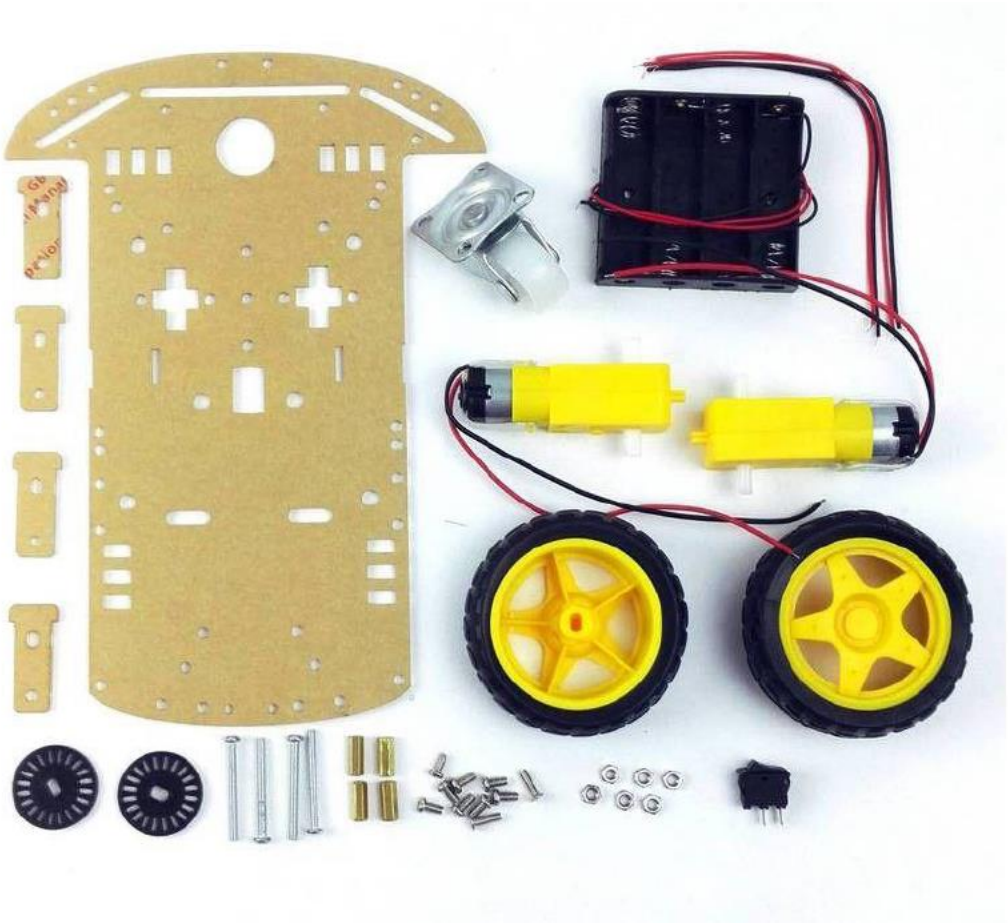
Stuck?

READ THE FAQS AT THE END

Building The Chassis

Assembly

Once you've identified all the components, grab the bag containing the parts needed to build the buggy's structure. All these items should be in the same bag, and it should include the following:



Next, take the chassis and peel off the protective film to reveal the transparent surface. It looks much cooler that way!



Before

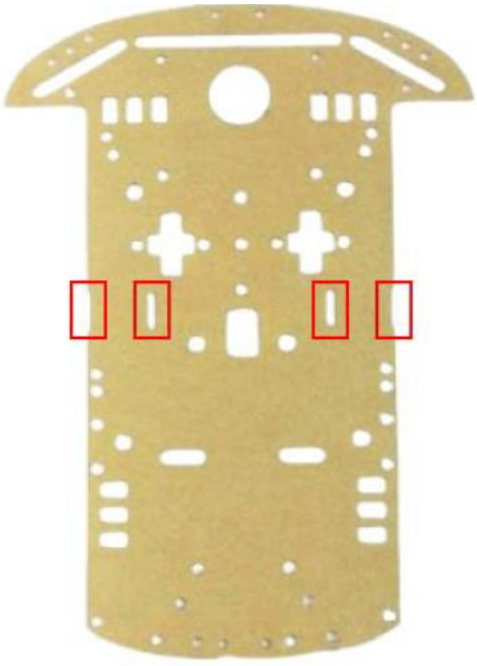


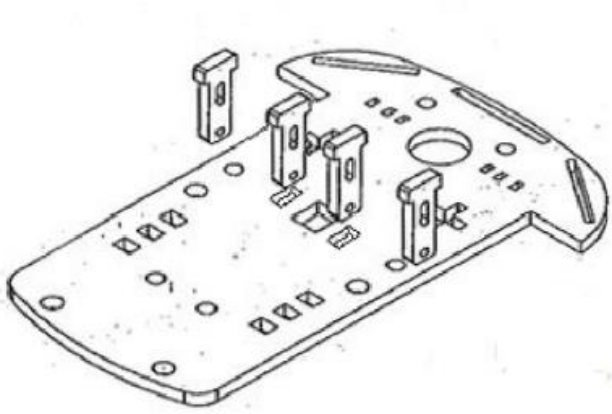
After

If you like, you can also peel the protective film off the T-shaped connectors for a cleaner look.

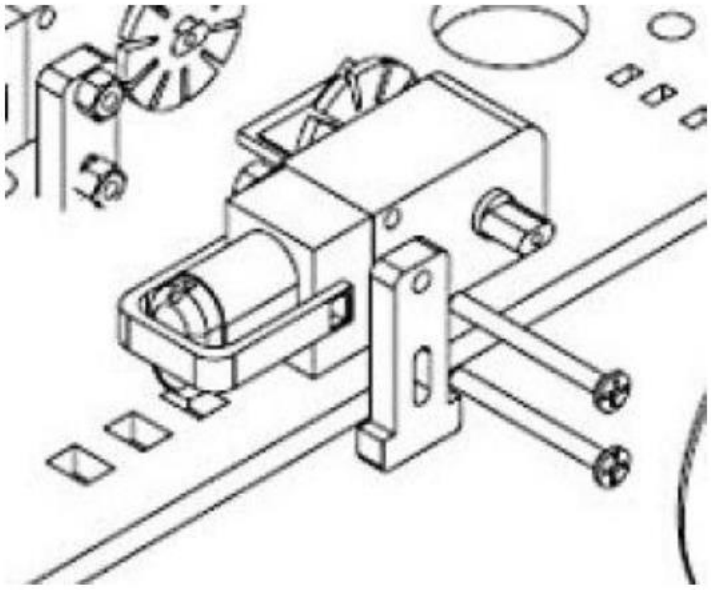


Now it's time to attach the motors. Take the T-shaped connectors and insert them through the holes shown in the pictures.

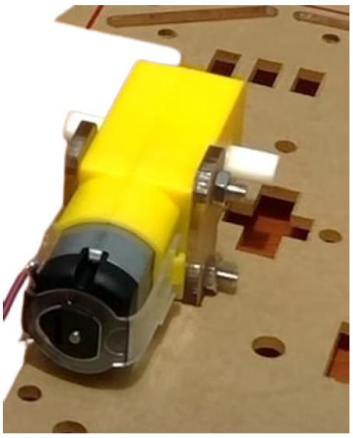


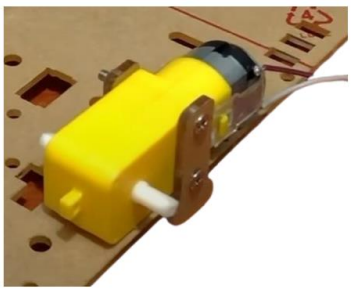


Next, flip the chassis upside down and place the motors between the T-shaped connectors. Once you've done that, use two M3 x 30mm bolts along with two M3 nuts to secure each motor to the chassis, as shown in the picture. Repeat this process for each motor on both sides.



Each motor should look like this:



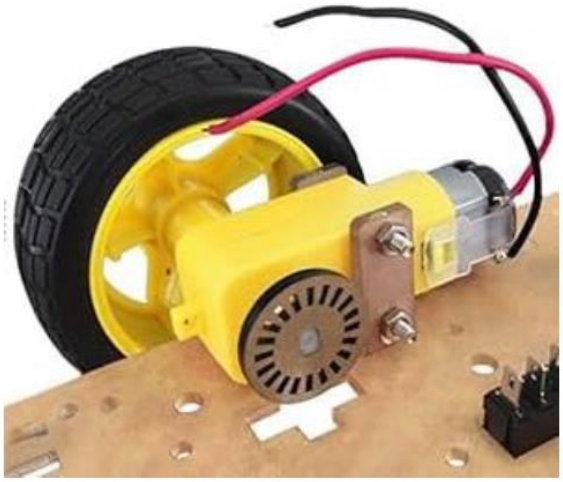


Be sure to tighten the bolts carefully, without overtightening, as the plastic is quite fragile. Try to avoid visible cracks, but also ensure the motors are securely in place without any wobbling. This will help your buggy move straight when you test it. Check and tighten the bolts periodically, as they can easily become loose over time.

If you'd like, you can also add the encoder wheel to the motor shaft on the side opposite the wheels. These are optional unless you plan to implement motor control using optical encoders, which you can explore in Buggy Plus during the second semester. For now, there's no real need to worry about it. Motor control is an advanced topic involving feedback loop control to ensure both wheels spin at the same rate. While it might sound simple, it’s actually quite tricky. If you want your buggy to be extremely precise, motor control would be essential, but it's beyond the scope of this beginner guide. If you're curious to learn more, feel free to explore resources online or ask the Buggy team for more information.



Encoder Wheel



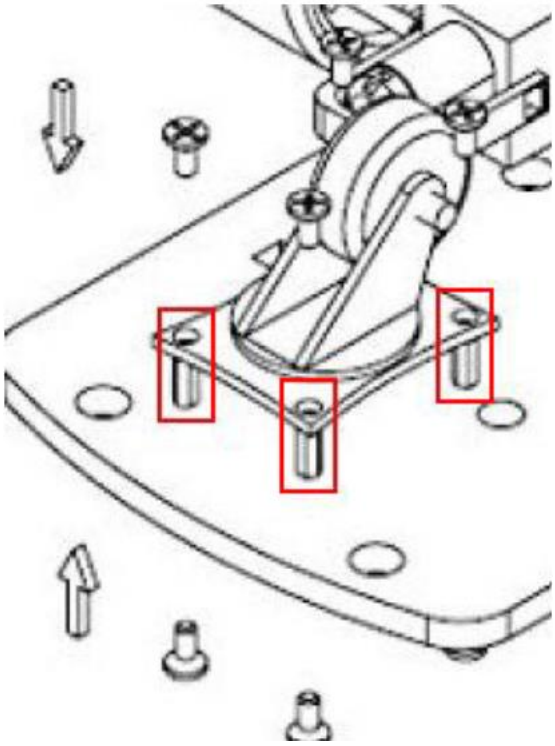
Now it's time to mount the caster wheel:



For this, you'll need four hex standoffs:



You'll also need eight M3 x 6mm bolts. Then, you can attach the caster wheel as shown in the picture

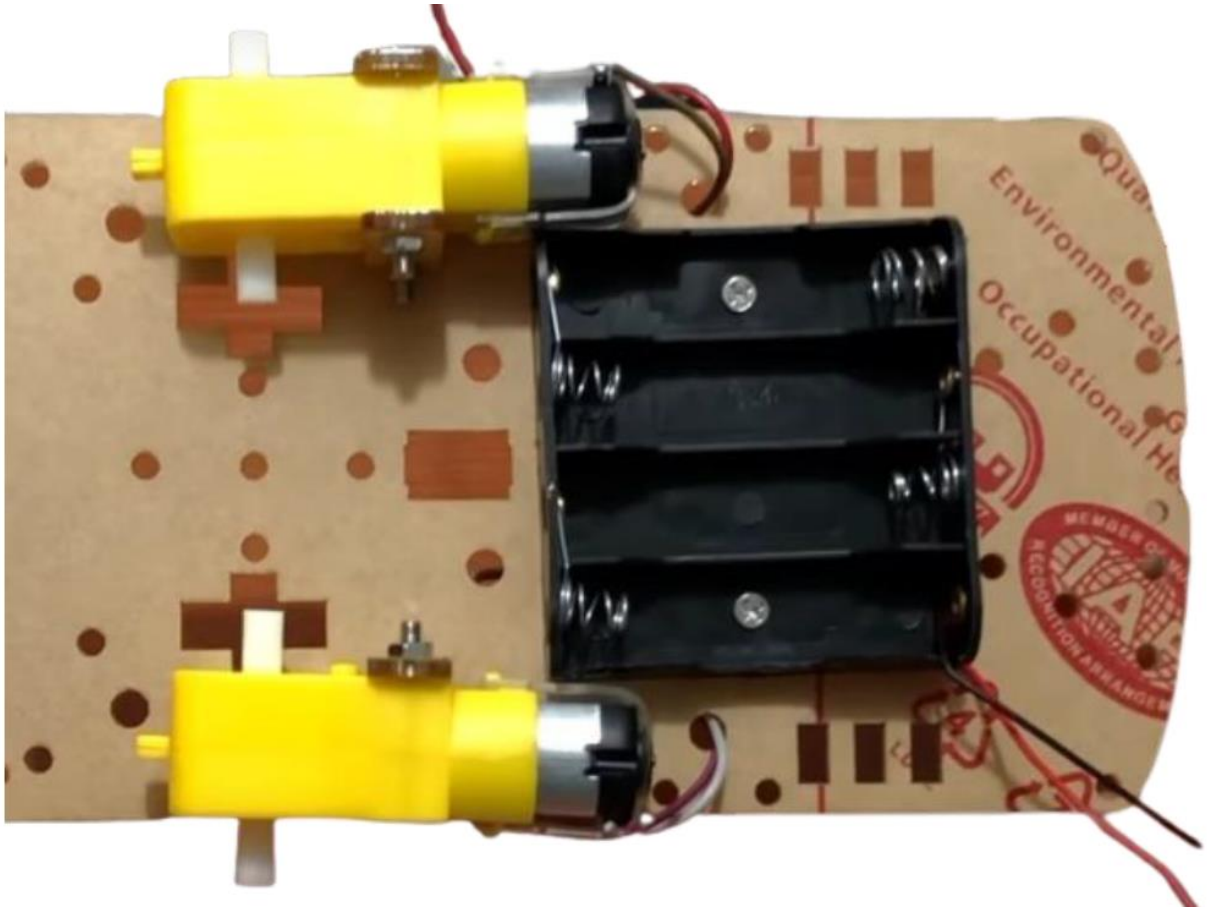


The objects highlighted in red are the hex standoffs.

By now, your buggy should be looking something like this:

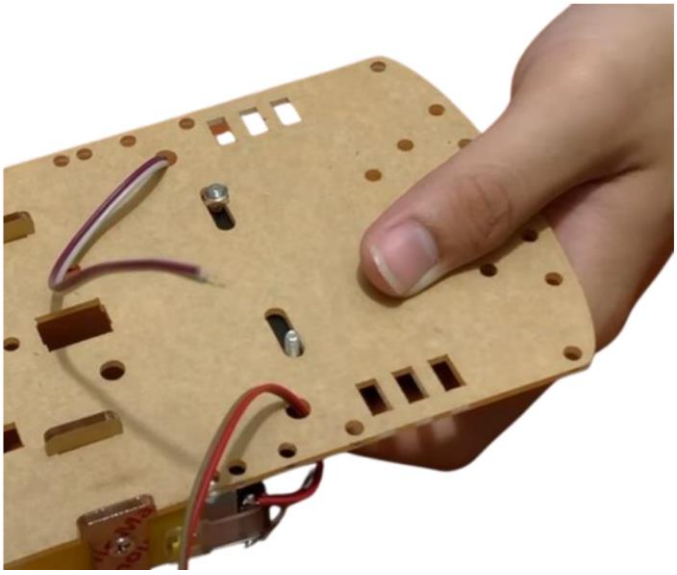


Next, you need to place the battery pack. The best position for it is likely this one:



As you can see, the battery pack should be positioned between the two motors at the bottom of the buggy. In the photo, the pack is secured using M3 x 10mm bolts and M3 nuts.

The top view shows where the bolts are placed:



Alternatively, you can attach the battery pack using hot glue for a quicker and easier solution.

I suggest attaching the battery pack to the bottom of the robot to leave space for other electronic components on top, making wiring easier. However, having the batteries upside down may cause them to fall out occasionally. While it's more of an annoyance than a major issue, if you notice it happening frequently, you might want to secure the batteries more firmly. Otherwise, your buggy could stop mid-competition if a battery falls out.

It's now time for the most satisfying part of the assembly: attaching the wheels!

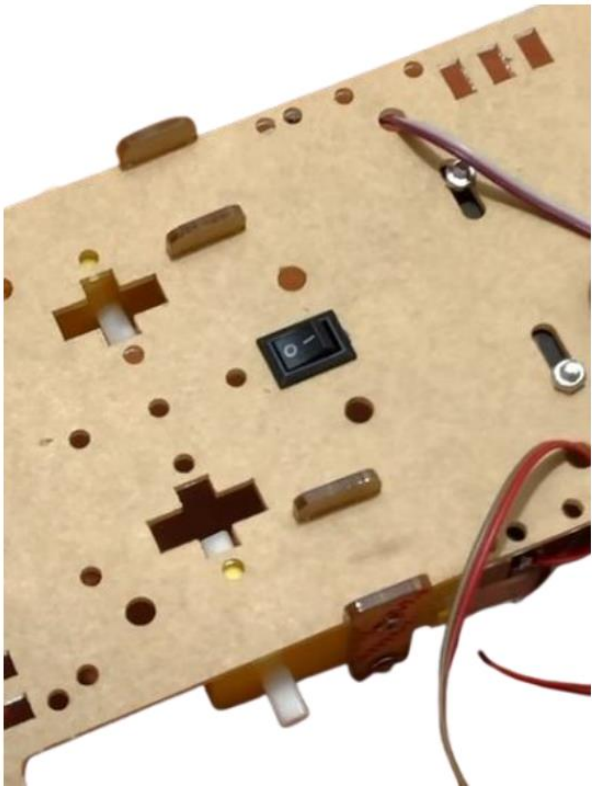


By now, your buggy should look something like this:

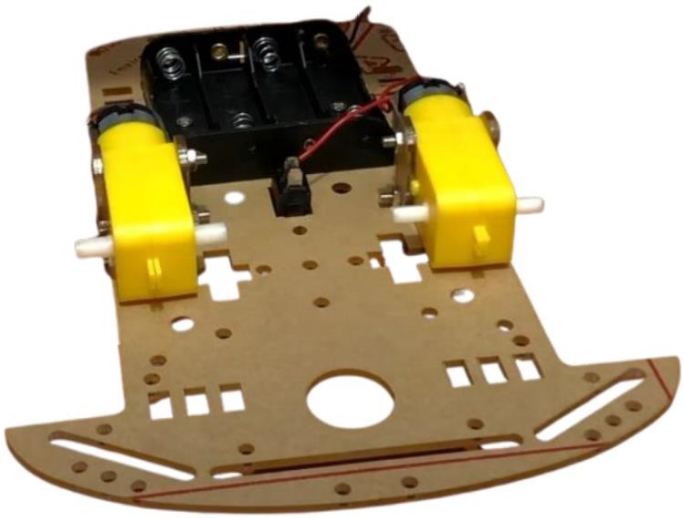


However, the battery pack should be positioned at the bottom rather than the top.

It’s now time to place the rocker switch. You’ll notice a rectangular cutout in the middle of the chassis designed for it; simply position the switch there.



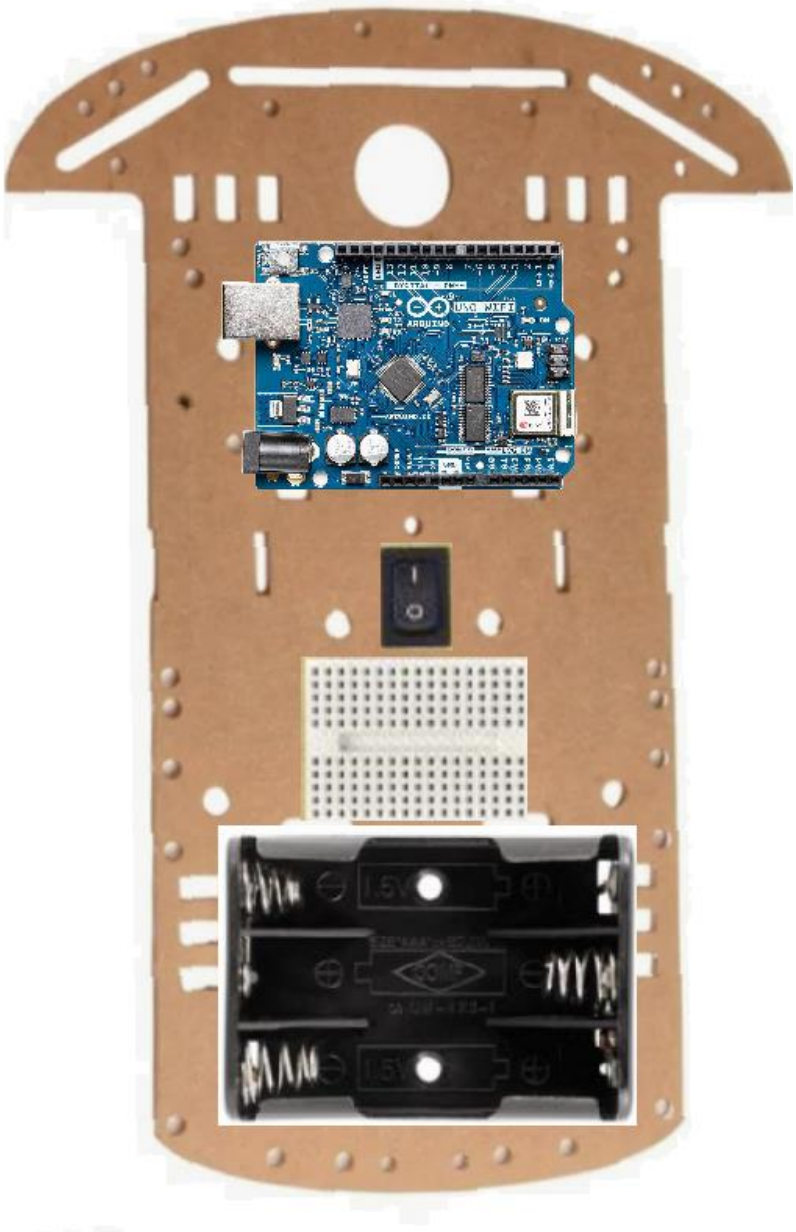
Top View



Bottom View

The final step in the building process is to attach the remaining electronics on top of the chassis.

To do this you can follow the following positioning:

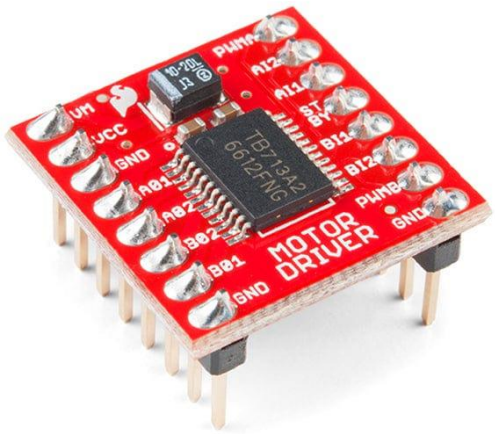


You can arrange your chassis in any way you'd like. In fact, if you wanted to you could try add a second layer! However, I recommend putting the uno not too far from the breadboard though as there will be a lot of wire connections between the two. Longer wires could get tangled in the wheels.

It may be easier if you attach the battery pack later, because we're going to have to solder it to the rocker switch.

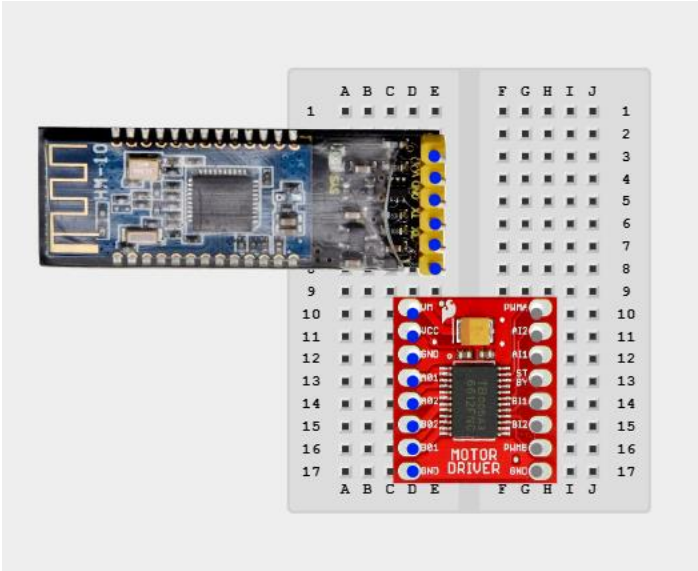
Make sure the rocker switch is in place, securing it with hot glue if necessary.

Finally, let's add your electrical components onto your breadboard. You should have a HM-10 Bluetooth module (right) and motor driver (left)





Attach these onto your breadboard like so.



NB: Might be an idea the Bluetooth module later. It will just get in the way for now.

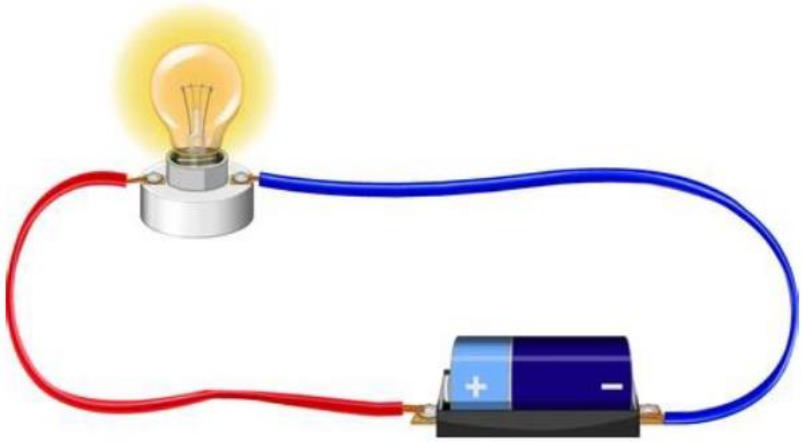
Congratulations! You should now have successfully built the structure of your buggy. If you followed the instructions correctly, your buggy should be sturdy and well-constructed. However, as you continue the process, be prepared to adjust and fix various components: tightening screws and applying plenty of hot glue will likely become part of

your routine. Great job on establishing a solid foundation for your buggy; now it’s time to move on to wiring the components together!

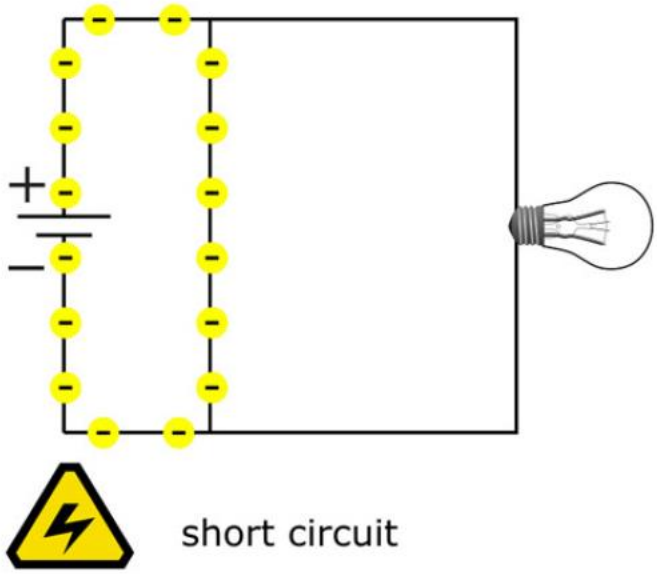
Powering the Buggy

Note: At the beginning of the session, we will distribute wires to each group. However, if you need additional wires at any point, feel free to approach us and collect more. There are plenty available for everyone.

When you connect a component to a battery, current flows from the positive terminal to the negative terminal. This current is carried by electrons moving through the circuit, traveling from a point of higher electric potential (voltage) to lower electric potential, similar to how a ball falls from higher to lower gravitational potential. For current to flow, there must be a closed path between the two terminals of the battery and components through which the current can pass, enabling the circuit to function.



Note that if there are no components in the circuit, there will be no resistance to oppose the flow of current, causing an excessive amount of current to flow. This is known as a short circuit, which can lead to overheating and potential damage to components, or in the worst-case scenario, even flames and explosions. Therefore, it's important to avoid short-circuiting the components.

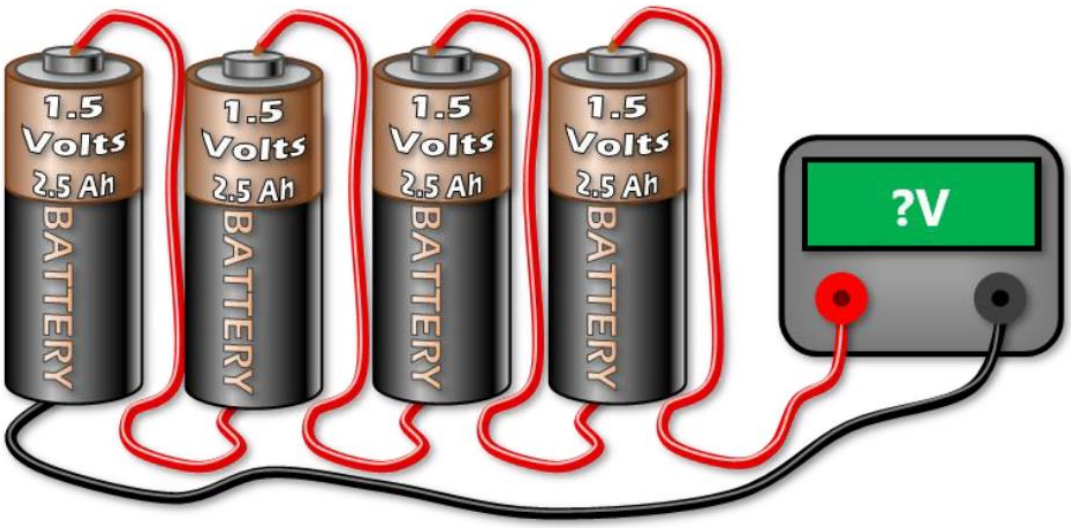


Every battery has a voltage rating. Specifically, the AA batteries provided to you have a nominal voltage of 1.5V (Volts), which is the voltage they have when new, which will decrese as you use them over time. You can think of voltage as the potential that pushes the current through the circuit: the higher the voltage, the more current it will drive.

Each component in a circuit is designed to handle a specific amount of voltage and current. For example, the HM-10 can be powered with 3.3V, and supplying it with more voltage may damage the component, even though it has a voltage regulator that should prevent it. Therefore, it is important to power it with exactly 3.3V. On the other hand, the Motor Driver can handle up to 12V to drive your DC motors. You'll need to connect the Motor Driver to the DC motors in order to control them effectively.

Keep in mind that, at this stage, you don't actually need the batteries since you're still in the process of wiring the buggy. However, if you'd like to have them now, feel free to come and ask for them.

The first thing you need to do is understand how to power your board. You'll be placing 4 batteries in the battery pack, so how many volts should you expect to get from it? Keep in mind that the batteries in the pack are connected in series, which means you can sum up their individual voltage ratings to determine the total voltage output of the battery pack.



Can you directly apply this voltage to the HM-10 Bluetooth Module?

As you've likely noticed, it's best not to connect this voltage directly to the HM10. If you have any doubts, feel free to ask someone from the buggy team to explain to you why.

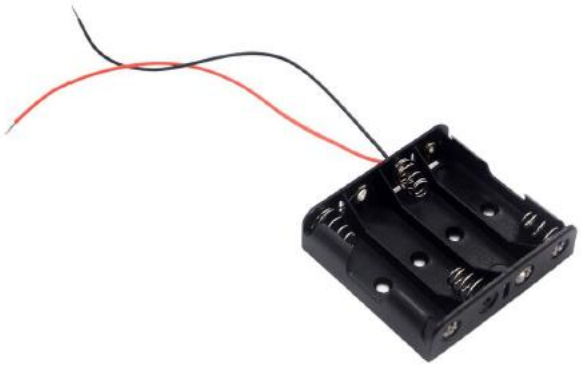
However, it is acceptable to supply this voltage to the motor driver, which is what we will do. How much additional voltage can the motor driver handle?

Remember, this guide is here to assist you if you're feeling lost, but the decisions are ultimately up to you. Consider how a higher voltage correlates with increased power to the motor, potentially leading to greater speed. If you're feeling creative, think about ways to reach that 12V limit to power the motor driver.

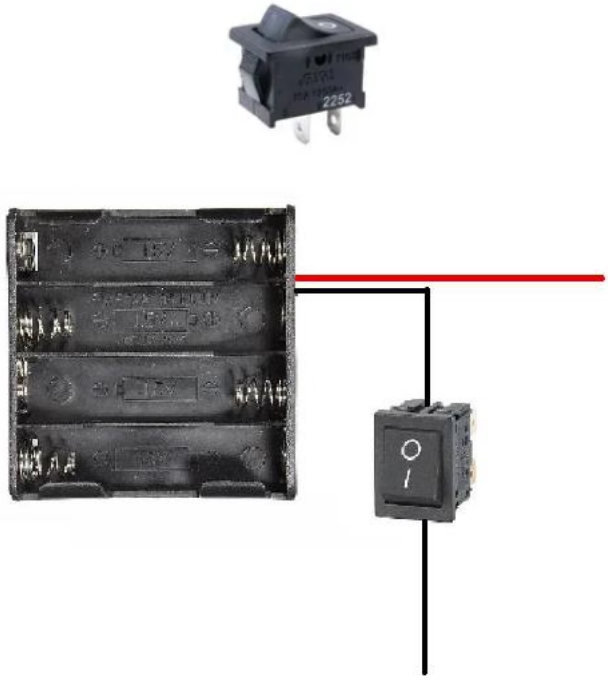
How can you achieve that voltage? What do you need? How much faster can you get?

It's your choice how to design your buggy, and ultimately, whether you want to push its limits to win the competition. If you think you need any additional components, feel free to ask for them. However, remember that there is no immediate need for extra parts, and if you prefer, you can continue following this guide without worrying about increasing the voltage.

For now, the closed circuit we want to create includes a battery pack with positive and negative terminals, represented by the red and black wires, respectively. Current will flow from the battery through the red wire around the circuit and then return to the battery via the black wire, which serves as the ground (GND) or low potential.



Next, we want to connect this to the motor driver and close the circuit. However, there's a missing component we should consider first. When the circuit is closed, the buggy will be powered, and when it's open, no current will flow, meaning the buggy won't be powered. Since we don't want our buggy to be always powered, we need a reliable and easy way to open and close the circuit without having to solder the wires every time. To achieve this, we need to incorporate a switch into the circuit.



I think the best way to do this is to solder the wires to the switch pins, like shown in the above diagram. If you don't know how to solder, I recommend attending our **Soldering Workshop**. Otherwise, feel free to continue following this guide. Ensure that the second cable, the one running from the switch to the breadboard, is equipped with male pins for easy connection to the breadboard.

Here you can see the difference between male and female jump wires.



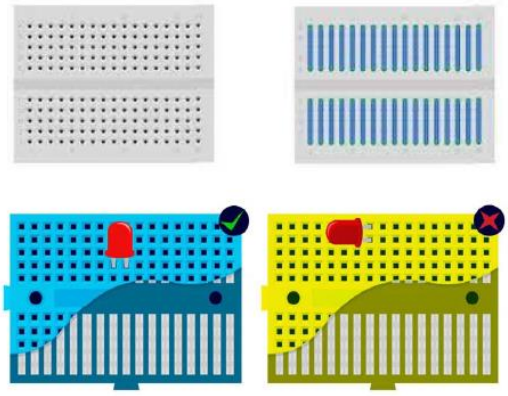




You also want to ensure that the two terminals of the switch don’t touch each other. If they do, the current will flow continuously, and you won’t be able to open the circuit by flipping the switch. This would mean you'd have to remove the batteries from the pack every time you want to unpower the buggy, which is far less convenient.

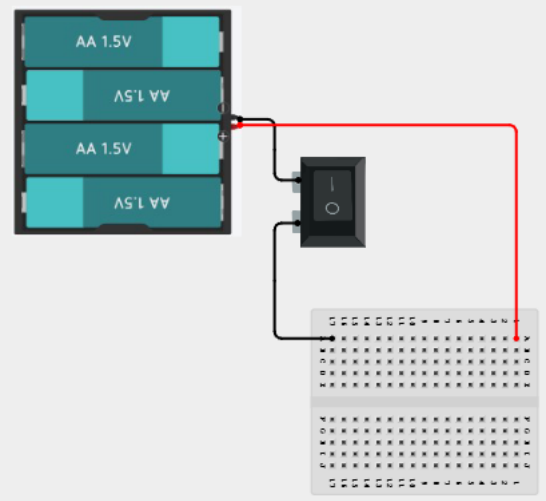


As a refresher, here's how a breadboard works:



Each column on the breadboard is connected internally, so if you need to connect two wires together, you don’t have to solder them. Instead, you can simply plug them into the same column, and they will be connected.

To make your life easier, I recommend creating positive and negative voltage rails on your breadboard. Connect one end of the wire into a column on the breadboard so that if you need to connect any component to ground (GND) in future, you can easily plug their wires into the same column. This will connect them to the battery's ground, effectively closing the circuit. Do the same for the positive supply voltage as well. If you have any doubts, feel free to ask someone from the buggy team. This method is helpful when connecting multiple components to ground, as the Arduino has limited ground ports, and you’ll need more than those provided. This is why breadboards are so useful.



Introduction to Arduino

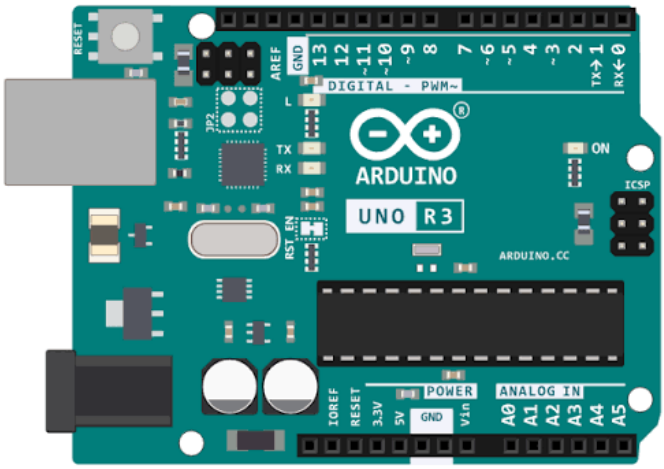
What is an Arduino?

Before continuing, I would like to introduce you to **Arduino**.

An Arduino is a small, programmable electronics board that can be used to control and interact with the world around it. It can receive information from sensors and other components, process that information using code, and control devices such as **motors, LEDs, and servos**.

For our buggy, the **Arduino Uno** acts as the brain of the robot. It receives commands from the remote control and sends instructions to the motor driver, telling the motors when to move, which direction to turn, and how fast to run.

Arduino is designed to be beginner-friendly, making it a great way to learn the basics of **programming, electronics, and robotics**.



Installing the IDE

IDE stands for Integrated Developing Environment and is essentially the application where you'll write your code, debug it, and upload it to the Arduino. You can download it from this link: https://www.arduino.cc/en/software.

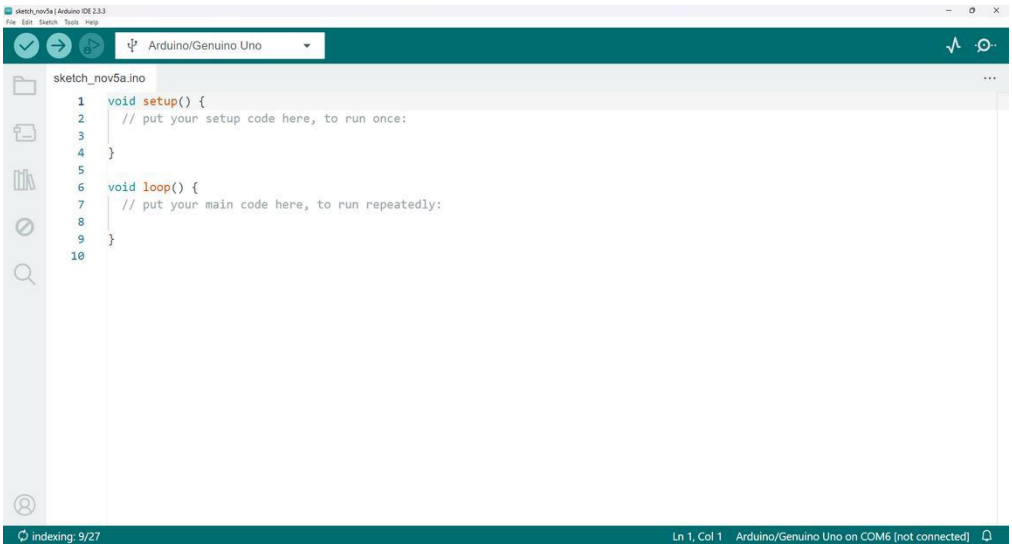
Downloads



On the left side, select your operating system and click to download the latest version.

It's best to use your own computer for programming the Arduino rather than a university computer, as you may run into issues when connecting the Arduino via USB. University computers often have restrictions on USB access for security reasons, which could prevent you from directly accessing the Arduino and potentially lead to complications in uploading code.

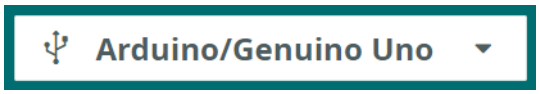
Once you've downloaded it, open the program, and you should see an environment that looks like this:



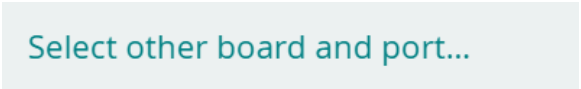
Next, save the file you're working on. You can do this by either going to File > Save As or pressing CTRL+S (or Command+S on Mac). Give the file a meaningful name, like “Amazing_Code_Buggy_Project_V1”, including a version number helps you keep track of updates. Save it in a location you can easily find; consider creating a new folder just for your buggy’s codes. From now on, save regularly by pressing CTRL+S (or Command+S on Mac) or going to File > Save to avoid losing any progress.

Next, take the black USB cable and use it to connect the Arduino to your computer.

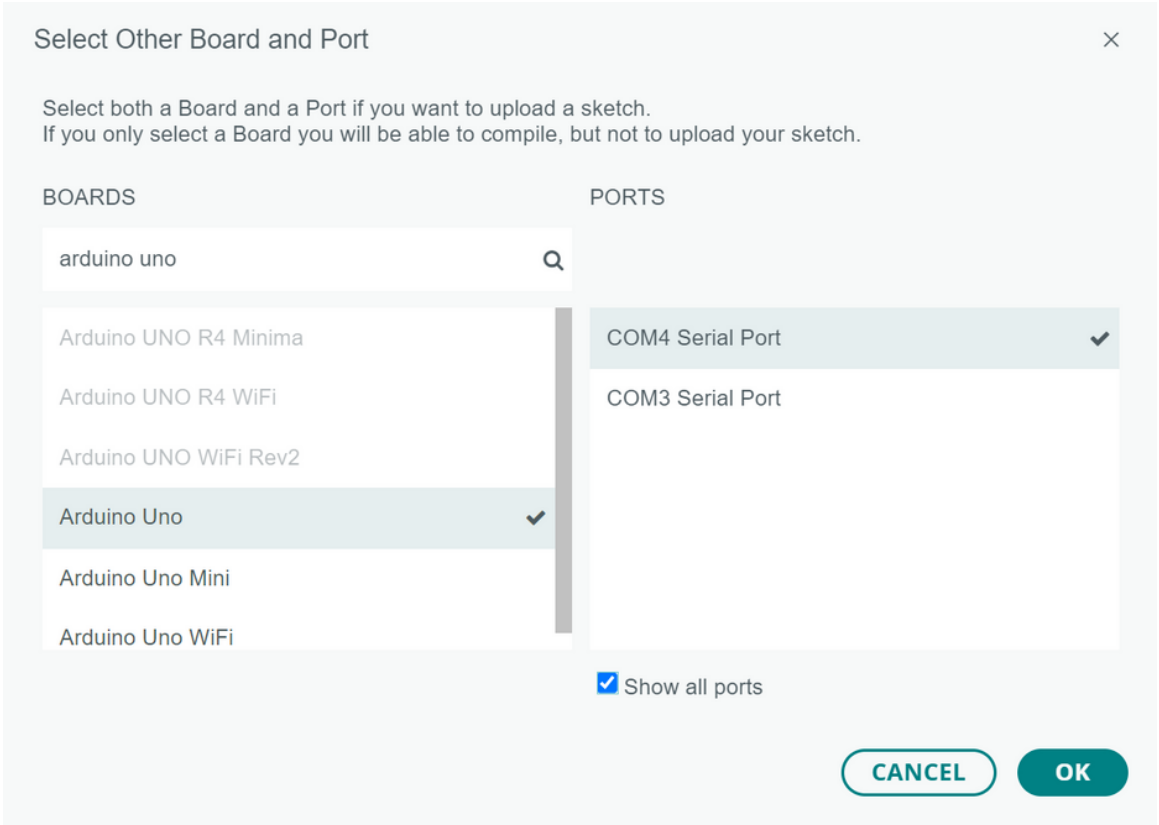
Next, click on this drop-down menu:



Then click:



Then you will get to the following screen:



Look for the Board section and select Arduino UNO from the list. This step is important because we’re using the Arduino UNO, so the IDE needs to know which board you’re

programming. Next, go to the Port section, and select one of the COM ports displayed. Ideally, select the port where your USB cable is connected to the Arduino. If you're unsure which COM port it is, just choose one of the options. You can test and switch it later if needed. Once you've selected the board and port, click OK to confirm.

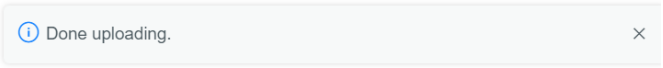
To run your default program, simply click the arrow button located at the top of the Arduino IDE. This button, often referred to as the Upload button, will compile your code and upload it to the Arduino board. Once you click it, the IDE will start the process of checking your code for errors. If everything is correct, it will then transfer the code to your Arduino. You'll see a series of messages in the status bar at the bottom of the IDE indicating the progress of the upload.



If you just want to test if your code is correct without uploading it to the Arduino, you can click the Checkmark button (located next to the Upload button). This will compile your code and check for any errors, but it won't upload the code to the Arduino. If there are any mistakes in your code, the IDE will display error messages at the bottom of the window, helping you identify where the issue is. This allows you to catch mistakes before attempting to upload the code to the board.



If the code uploads successfully to the Arduino, then everything is working fine, and you should see this:

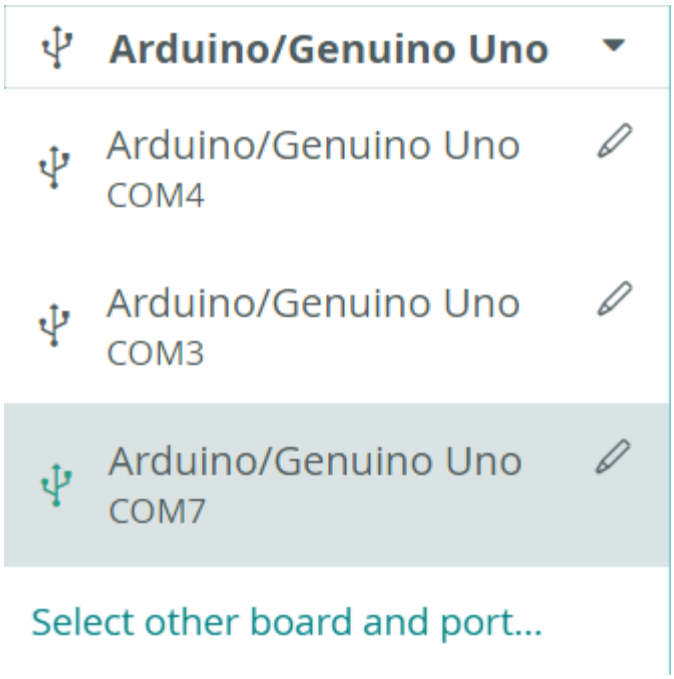


This means that you have successfully connected the Arduino to the COM port you selected earlier. However, if you see the following error:

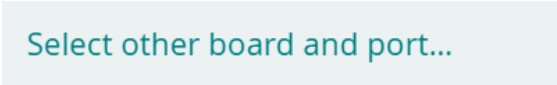
```
avrdude: ser_open(): can't open device "\\.\COM4":
```

This means that the Arduino is connected to a COM port different from the one you selected.

Typically, the IDE will suggest the correct port, and it will appear in the dropdown menu as shown below:



You can try one of the given ports, and usually, one of them should be correct. Test them and see which one works. If it still doesn't upload your code, you can go back to this:



And try selecting another COM port, then repeat the procedure until you find the one that matches. Alternatively, you can physically attach the cable to another USB port until you find the one that works.

Setting up the pins

Let's now take a look at the default code you have been provided with:

```
1  void setup() {
2    // put your setup code here, to run once:
3
4  }
5
6  void loop() {
7    // put your main code here, to run repeatedly:
8
9  }
```

In the Arduino IDE, the void setup() function is executed only once when the Arduino is powered on or reset. It is used to set up the initial conditions for your project. This could

involve configuring pins as either inputs or outputs, initializing communication with other devices, or starting any processes that need to be run before the main loop starts. After setup(), the void loop() function runs continuously. This is where the main logic of your program happens, and it will execute over and over again, allowing you to constantly monitor inputs, make decisions based on those inputs, and control outputs like LEDs or motors.

Variables are essential for storing and manipulating data in your Arduino program. Think of a variable as a labelled container where you can store information such as numbers, text, or other values. To use a variable, you first need to declare it. This is done by specifying the type of data it will hold (such as int for whole numbers, float for decimal numbers, or String for text) and giving it a name. For instance, if you want to store the number 9 in a variable called ledPin, you would declare it like this: “int ledPin = 9;”. This means that ledPin now refers to the number 9, which corresponds to digital pin 9 on the Arduino board. You can now use ledPin in your code instead of writing the number 9 every time you refer to that pin.

Using variables this way makes your code more readable and easier to update. For example, if you want to move the LED to a different pin, you only need to change the value of ledPin in one place, and the rest of the code will automatically follow this change.

Pin initialization refers to setting the pins on your Arduino board as either input or output, which tells the Arduino how each pin will behave. You use the pinMode() function to do this, passing two arguments: the pin number and the mode (INPUT or OUTPUT). In the void setup() function, you would initialize pins that will be used for communication with external devices like LEDs, buttons, or

sensors. For example, if you want to use pin 9 to control an LED, you would write “pinMode(ledPin, OUTPUT);”. This tells the Arduino to use pin 9 as an output, so it can send power to the LED. If you want to use pin 2 as an input to read a button, you would write “pinMode(2, INPUT);”. This configuration allows the Arduino to read the button’s state, whether it’s pressed or not.

To combine both concepts, you can declare your variable and initialize your pins together in your Arduino sketch. Here’s how it might look:

```
int ledPin = 9; // Variable for the LED pin

void setup() {

  pinMode(ledPin, OUTPUT); // Initialize the LED pin (pin 9) as an output
  pinMode(2, INPUT);       // Initialize pin 2 as an input

}
```

Note that since you are programming in C++, you need to place a semicolon (“;”) at the end of every line.

Now, it's time to set up all the pins you’ve connected to the Arduino so far. Be sure to use meaningful variable names for your pins to make your code more readable, easier to understand, and ultimately simpler to debug. Carefully consider whether each pin should be set as an input or output. Remember that you only need to configure digital or analog pins, not power pins. Getting this step right is crucial, as it will save you a lot of time and frustration during the debugging process.

Testing your Arduino – LED blink

Before we start building the buggy, let's make sure your Arduino is working correctly. We can do this by making the small **built-in LED** on the Arduino blink.

Connect your **Arduino Uno** to your computer using a USB cable.

Open the **Arduino IDE** and make sure you have selected the correct:

- **Board:** Arduino Uno
- **Port:** The port your Arduino is connected to

If you are unsure which port to select, refer back to the Arduino IDE setup instructions.

Open a new Arduino sketch and enter the following code:

```
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(1000);

  digitalWrite(LED_BUILTIN, LOW);
  delay(1000);
}
```

Don't worry if you don't understand everything yet – we'll cover how Arduino code works later in the guide.

Click the **Upload** button in the Arduino IDE.

The IDE will first compile your code and then upload it to the Arduino. Wait until the IDE indicates that the upload has completed.

Look at the small LED labelled **L** on your Arduino Uno.

If everything has worked correctly, the LED should light up for a second and then turn off for a second continuously.

If it doesn’t work, check:

- Is the USB cable securely connected?
- Have you selected **Arduino Uno** as your board?
- Have you selected the correct **Port**?
- Does the Arduino appear as connected to your computer?
- Did the IDE report any errors when uploading?

If you're still having trouble, ask the Buggy Team for help before moving on. We’re more than happy to help.

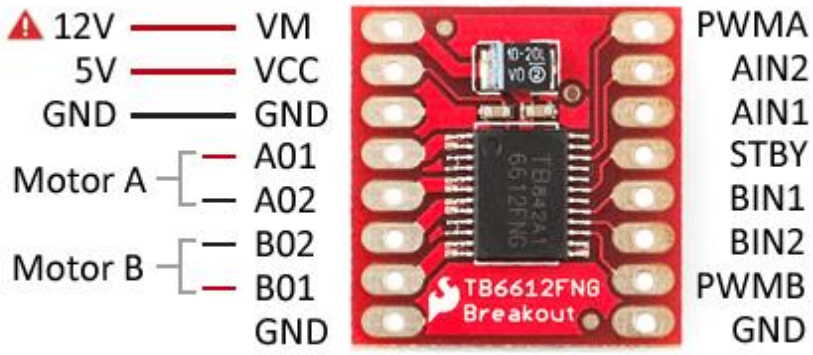
Motors

Introducing the motor driver

A motor driver is a component that allows an Arduino to control motors. Since an Arduino cannot supply enough current to power DC motors directly, the motor driver acts as an interface between the Arduino and the motors, allowing it to control their speed and direction.

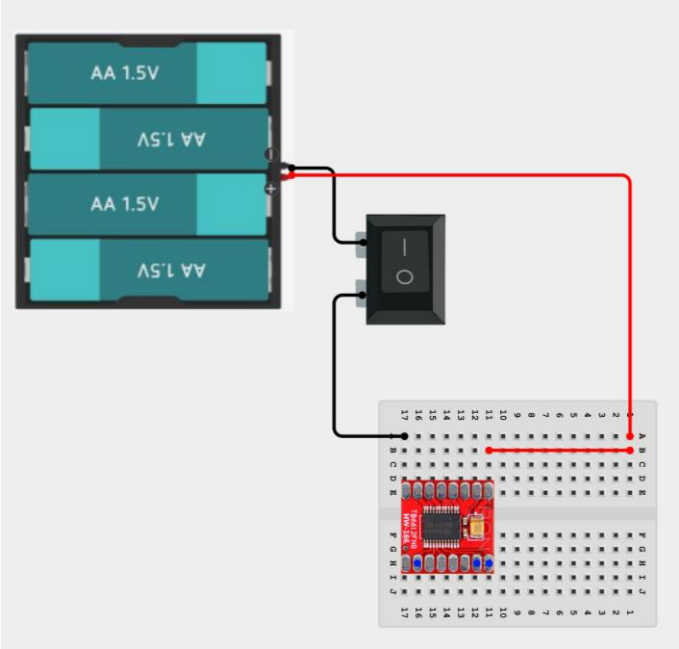
The TB6612FNG is a dual motor driver, meaning it can control two DC motors independently. It receives control signals from the Arduino and uses a separate power supply to provide the higher current required by the motors. This allows the buggy to move forwards, backwards, and turn by controlling the two motors individually.

Below you should see a diagram with all the pins of the motor driver. Each pin serves a different purpose.



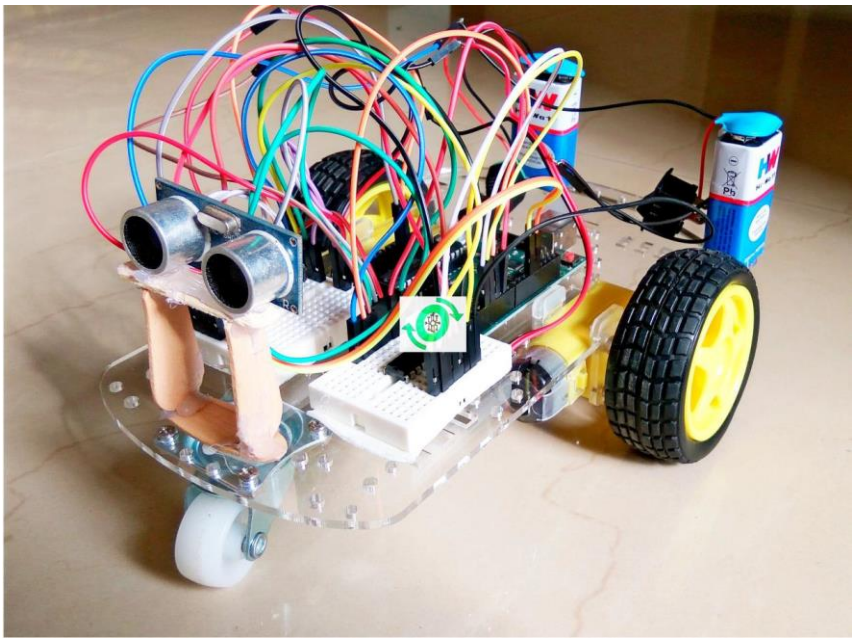
- PWMA/B controls the speed of motor A/B.
- AIN1/2 and BIN1/2 control the direction of motor A and B respectively.
- A01/2 and B01/2 output the voltage to the motors.
- STBY is the master enable/disable for the driver (If STBY is in LOW state, the chip is essentially off).
- VCC you connect to the Arduino 5V power. This is the logic power.
- VM is motor power supply. This can take up to 12V. This is for powering the motors.
- GND is ground and must be connected to complete all the circuits. Note only use one of the available GND.

We're going to start by powering the driver. Begin by connecting VM and GND from the motor driver to the positive and negative power rails.



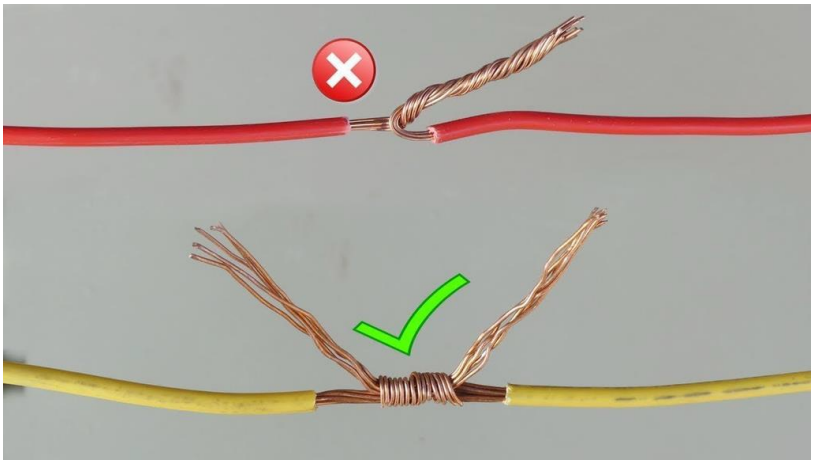
Whenever you connect wires, try to follow a consistent color scheme and keep your wiring as neat and organized as possible. This will not only make your buggy look much better, but it will also simplify troubleshooting when you encounter errors. An organized setup saves you time and effort in identifying the cause of a problem, whether it's related to coding, wiring, or a broken component. For now, aim to make it easy to find and fix any wiring mistakes by keeping everything tidy.

Don’t be this guy:



If any of your cables are not long enough, you can always connect two wires together by soldering them. After soldering, cover the connection with electrical tape for safety. If you create a blob of solder, don’t worry; it will allow current to flow more easily and won’t affect the circuit's performance. However, ensure that the connection is covered properly to avoid electric shock and to maintain a strong solder joint, as you don't want to constantly repair it or risk it failing during the competition.

Make sure to twist the wires together before soldering for a stronger connection that can withstand some force. If the connection breaks, don’t be discouraged; making a strong solder joint between two wires can be challenging. You'll likely need to go back and fix it multiple times, but that's all part of the learning process!



Wiring the motors

Now let's get the motors connected into the system. The connection pads on the motors aren’t designed for jumper wires to plug into them, so we’re going to have to solder wires onto them.



You might need to remove the motor from the chassis to do this if it’s easier – sorry!



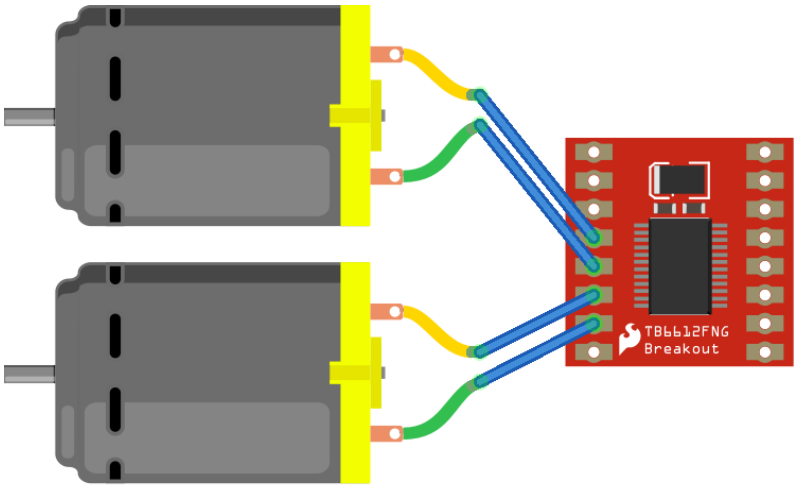
Once the motors are looking like the above you should find 4 screw terminals in your kit. These allow you to turn loose wire connections into pins that can easily be plugged into your breadboard. Just screw the top down once the wire is in place. Make sure to strip it a fair bit though to help the connection.



Alternatively, you could also cut a jumper wire and solder that to the motor, but this will give you less freedom over the length of the wire. In the first option you can always trim it down to exactly the required length at the end if you have to rearrange the parts on your buggy. It's your buggy though so you decide how you want to do it.



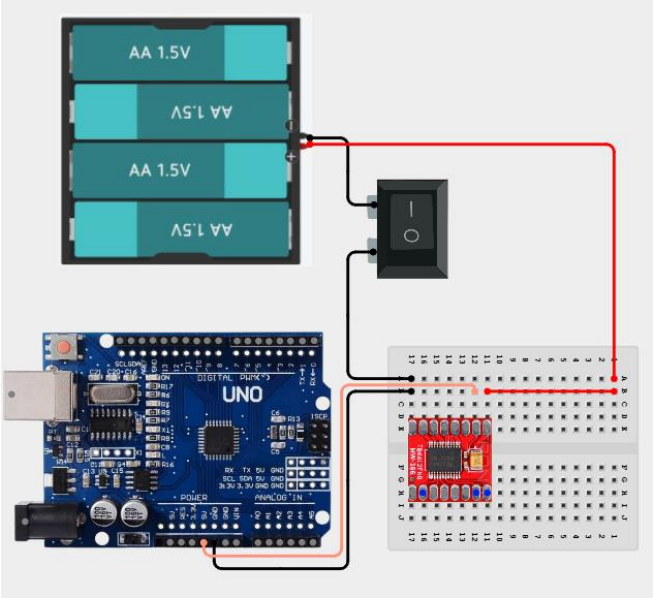
Do this for both motors and it should look something along the lines of this heavily simplified diagram.



Connecting the motor driver to the Arduino

Now that the motors are attached, let's get the driver connected to the Arduino so we can get the buggy moving.

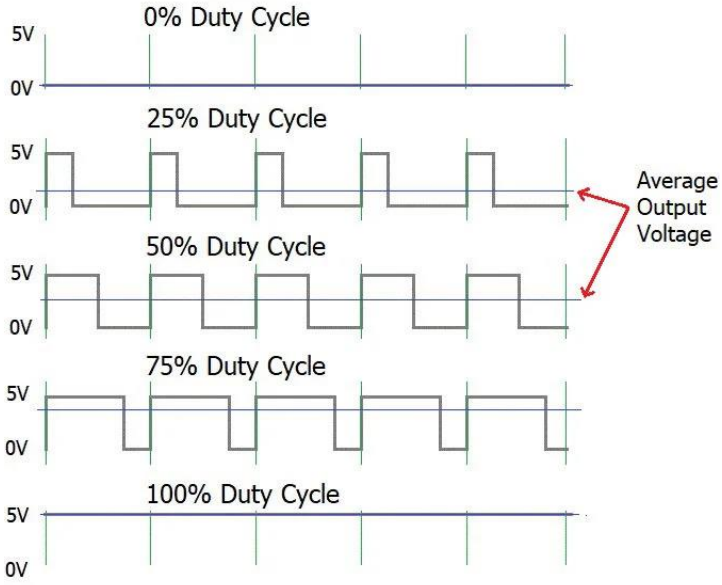
To begin with, lets connect the Arduino power supply to the motor driver's input. This is nice and simple. Just connect a wire from the 5V pin on the Arduino to the VCC pin on the motor driver, as shown below.

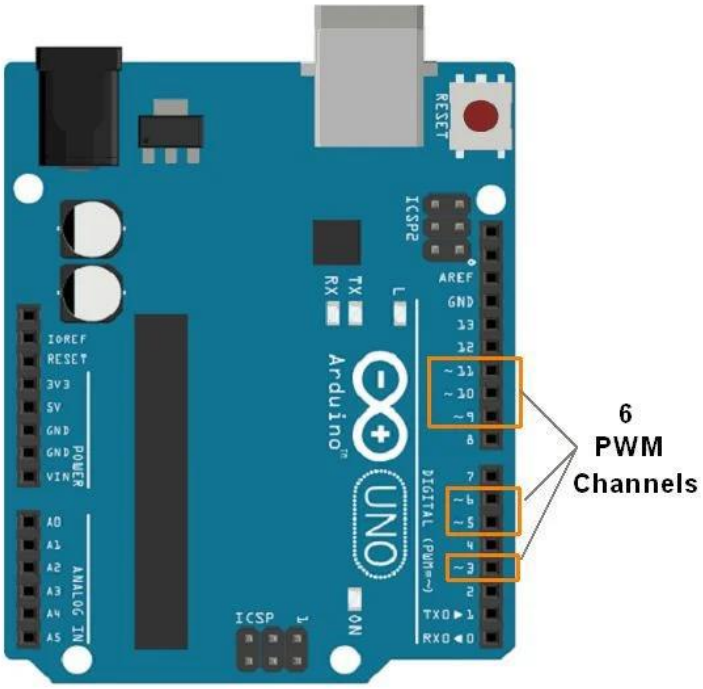


You should now have 3 sets of connections left to do between the driver and the Arduino

- The PWMA/B pins – controlling the speed of the motor
- AIN1/2 and BIN1/2 – controlling the direction the motor spins
- STBY – enables/disables the driver itself

For the most part you can allocate each of these to any of the digital pins 2-13 on the Arduino, except for the PWMA/B. For these ones, you must wire it to one of the respective PWM pins on the Arduino board. PWM stands for pulse width modulation, which is essential for controlling the speed of the motors. For those curious, it basically allows you to control the average voltage output from that pin (Refer to below Diagram).





Make sure to write down which pins of the motor driver you connected to which pins on the Arduino. This will make it easier for you to debug and identify errors in the future, especially if a wire comes loose. Additionally, it will help you easily associate the pins you're using with their corresponding functionalities when you start coding. Keeping track of this information will save you time and effort down the line.

Tip: To keep your wires neat and organised, use wires that are still bundled together. This will help prevent them from getting tangled or scattered around. Additionally, you can use tape or small bands to keep related wires together, ensuring a tidy setup that makes troubleshooting and maintenance easier.

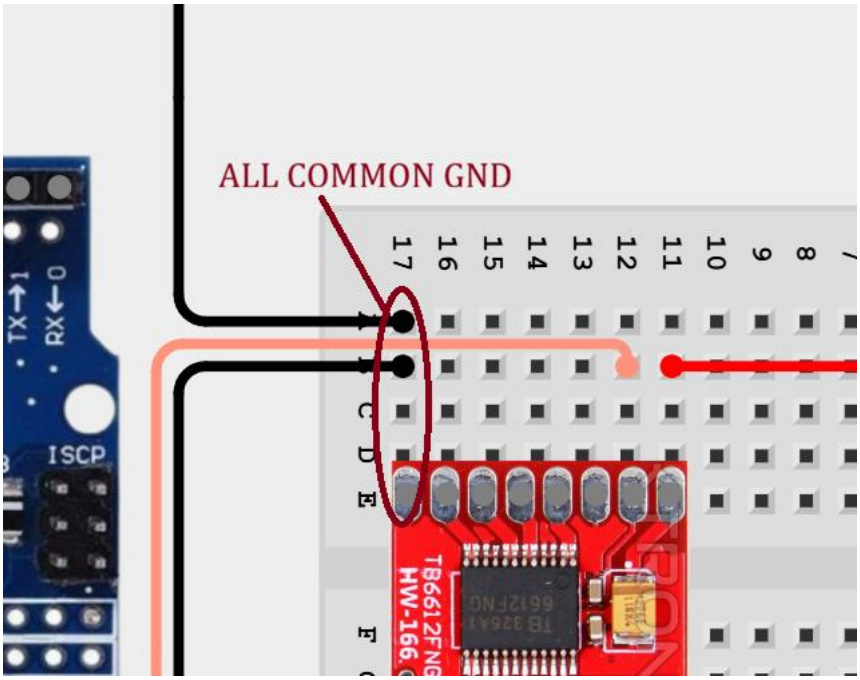
Powering the Arduino

So now we've built all the hardware to get the buggy moving! Only issue is there's nothing powering it.

Your buggy needs a power source to run the Arduino. For this project, we are using a **6V battery pack containing four AA batteries.**

The Arduino Uno can accept this voltage through its **VIN** and **GND** pins, allowing the board to regulate the voltage down to the level it needs.

ENSURE COMMON GROUND BETWEEN ARDUINO AND MOTOR DRIVER. Make sure they're both connected to the same GND pin on your Arduino Microcontroller. You can do this by utilising the breadboard channels.



If you've worked in Arduino before, you might've connected your power supply to the 5V pin, which works if the supply is exactly 5V, the nominal voltage of an Arduino. However, this is not the case here. If you remember from earlier, our batteries sum to 6V, which would fry the Arduino. Instead, we feed it to the VIN port which has a voltage regulator to bring the voltage back down to operating 5V. However, note that the nominal operating range for VIN is 7-12V, so you might experience voltage dips in your circuit. These can result in:

- Arduino resets
- Bluetooth disconnections
- Motors behaving unpredictably
- Other components malfunctioning

If you experience these, chances are it's a power issue. This is an ongoing battle for us in this project, but hopefully using fresh batteries will solve any of these issues.

Ideally, we would have separate power supplies for the Arduino and the rest of the components, but good luck convincing Fatima to fund buggy more.

However, please note that this is **your** project. If you think of a better way to do something, you can do it that way. There are many ways you can power your buggy, some of which will probably be more efficient than what I wrote here. This guide is only here to help you if you're stuck, don't be afraid to try new things and do stuff your way.

Also note by 'nominal voltage' I mean the range of voltages that should be input to ensure it works well.

Getting the buggy moving

You should have something that looks like this now

[image]

Great! If you've done everything correctly, your buggy should now be ready to get moving! But how do we do this? We could in theory blast it with loads of voltage and send it flying, but this is robotics society. We want to make it smart. So, we are going to use the microcontroller we've built into the system and programme it to do what we want – in this case, move forwards.

Enabling the Motor Driver

Before the TB6612FNG can control the motors, it needs to be taken out of standby mode. This is controlled using the **STBY** pin. Set STBY to HIGH:

```
digitalWrite(STBY, HIGH); // Enable the motor driver
```

If STBY is LOW, the motor driver will remain in standby and your motors will not run. Don't forget to make sure you have configured STBY as an output in your setup.

Controlling Motor Speed

The **TB6612FNG motor driver** is designed to control the speed and direction of two DC motors. It has separate inputs for each motor that receive signals from the Arduino.

For **Motor A**, the relevant pins are:

- **AIN1 and AIN2** – control the direction of the motor
- **PWMA** – controls the speed of the motor

For **Motor B**, the relevant pins are:

- **BIN1 and BIN2** – control the direction of the motor
- **PWMB** – controls the speed of the motor

The TB6612FNG also has a **STBY (Standby)** pin. This must be set to **HIGH** for the motor driver to operate.

To control the speed of the motors, you will use the `analogWrite()` function on the **PWMA** and **PWMB** pins.

`analogWrite()` sends a PWM (Pulse Width Modulation) signal to the motor driver. This allows you to control how much power is supplied to each motor and therefore adjust its speed.

The values you can use with `analogWrite()` range from **0 to 255**:

- 0 = 0% power
- 128 = 50% power
- 255 = 100% power

For example, you might choose a speed of **200** for the left motor and **150** for the right motor:

```
analogWrite(PWMA, 200); // Set left motor speed to 200
analogWrite(PWMB, 150); // Set right motor speed to 150
```

This means that the left motor will be instructed to run faster than the right motor.

Remember: PWMA and PWMB are variables that should have been assigned to the Arduino pins you selected earlier. Make sure you use PWM-capable pins for these connections.

Even if you set both motors to the same speed, you may find that one motor naturally runs faster than the other. This is completely normal! Differences between motors, wheels, batteries, and even the surface you're driving on can all affect how straight your buggy travels.

Once you have finished this section, try setting both motors to full speed and see whether your buggy travels in a straight line. If it does, you're in luck! However, if it veers to one side, you will need to experiment with different speed values for each motor.

For example, you might find that:

```
analogWrite(PWMA, 200);
analogWrite(PWMB, 180);
```

makes the buggy travel straighter than using the same value for both motors.

Getting your buggy to drive straight is one of the significant challenges of this project. It may seem simple at first, but finding the right balance between the two motors can take some experimentation.

You can set the motor speeds once in the `void setup()` function if you want them to remain constant throughout the program. Alternatively, you can change them whenever necessary in `void loop()` or within your own motor functions.

Keep in mind that your ideal speed values may change over time. Factors such as **battery discharge, floor surface, friction, and the weight of your buggy** can all affect its performance.

This is especially important when programming turns. For example, if your buggy needs to turn for half a second when the batteries are fully charged, the same amount of time may produce a different turn once the batteries have discharged.

Don't be afraid to experiment! Finding the correct values is part of the challenge and is one of the most important skills you will develop during this project.

Controlling Motor Direction

To change the direction of a motor connected to the TB6612FNG, you will use the `digitalWrite()` function. By setting the two direction pins for a motor to different combinations of **HIGH** and **LOW**, you can control which direction the motor rotates.

For **Motor A**, you can use **AIN1** and **AIN2**.

For example:

```
digitalWrite(AIN1, HIGH); // Set AIN1 to HIGH
digitalWrite(AIN2, LOW);  // Set AIN2 to LOW
```

This will cause Motor A to rotate in one direction.

If you reverse the two signals:

```
digitalWrite(AIN1, LOW);
digitalWrite(AIN2, HIGH);
```

The motor will rotate in the opposite direction.

The same applies for **Motor B**, but using **BIN1** and **BIN2**.

Which combination makes your motor rotate clockwise or counterclockwise depends on **how you have connected the motor wires**. Don't worry if your motor spins in the opposite

direction to the example, you can simply swap the motor wires or reverse the HIGH/LOW combinations in your code.

Depending on how you have configured the pins AIN1, AIN2, BIN1, and BIN2, you can control your buggy's movements, such as going forward, going backward, stopping, turning right, and turning left. It's crucial to keep track of which IN pins are connected to which motors to avoid any confusion, make sure not to swap the right motor with the left motor.

The **delay()** function in Arduino is a simple yet powerful tool that pauses the program for a specified amount of time, measured in milliseconds. When you use "delay(1000);", for example, the program will stop executing for 1000 milliseconds, which is equivalent to 1 second. During this pause, the microcontroller does not perform any other tasks or process any additional code.

This function is particularly useful when driving motors because it allows you to control how long the motors run or how long they remain in a certain state before changing their behaviour. For instance, if you want to move a motor forward for a specific duration, you can use the digitalWrite() function to set the motor direction, then follow it with a delay() to keep the motor running for the desired amount of time. After the delay, you can stop the motor or change its direction.

Now, go ahead and experiment with these functions to make your buggy move around. Great job on reaching this milestone!

Making Functions

To make your code more readable you now may want to start using functions.

Functions are blocks of code that perform a specific task and can be reused throughout your program. By using functions, you can avoid repetition, keep your code organized, and make it easier to understand. Instead of writing the same lines of code multiple times to make your buggy move in different directions, you can create a function for each action, which simplifies your main program and enhances clarity.

To create a function that tells your buggy to go forward, follow these steps:

Define the Function: Begin by defining the function at the beginning of your code. You can name it moveForward(). The function should include the code necessary to set the motors' input pins to make the buggy move forward.

```
void moveForward(){
  digitalWrite(AIN1,HIGH)
  digitalWrite(AIN2,LOW)
  digitalWrite(BIN1,HIGH)
  digitalWrite(BIN2,LOW)
}
```

Please note that this code has been written outside of the void setup() and void loop() functions.

Call the Function in the loop(): Now that you have defined your function, you can call it from the loop() function to make your buggy move forward. Just use the function name followed by parentheses.

```
void loop() {

  moveForward();  // Call the function to move forward
  delay(2000);    // Run forward for 2 seconds
  // You can add code here to stop the buggy or to call other functions

}
```

Please note that in this code, the buggy will move forward for two seconds and then repeat this behaviour indefinitely, as it is located within the void loop() function.

There are different ways to write the functions. You could make it move continuously instead of for set times.

Now it's time for you to create the additional functions that will enable the buggy to turn right, turn left, and stop.

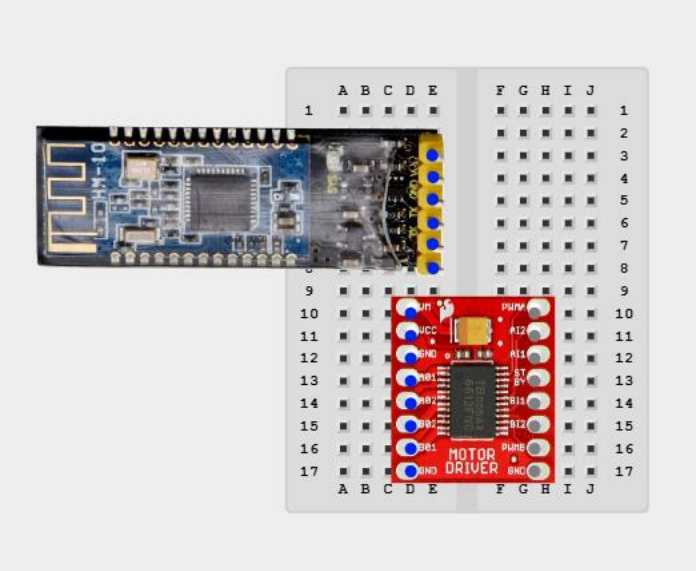
Additional Challenge: Navigating a square path with your buggy!

Once you've created these functions challenge yourself to make the buggy navigate a square path continuously. This experiment will demonstrate to you how imprecise the buggy can be during turns.

Implementing the Bluetooth Module

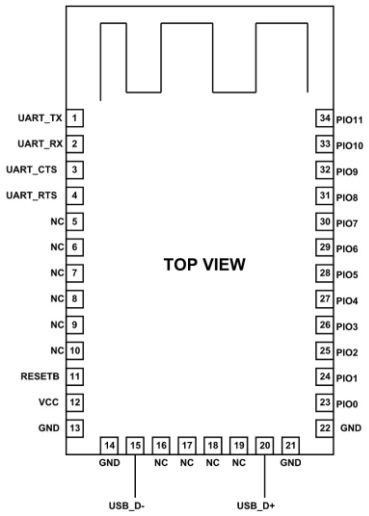
Adding the HM-10 on your breadboard

Your breadboard should currently look something like this but with many more wires.

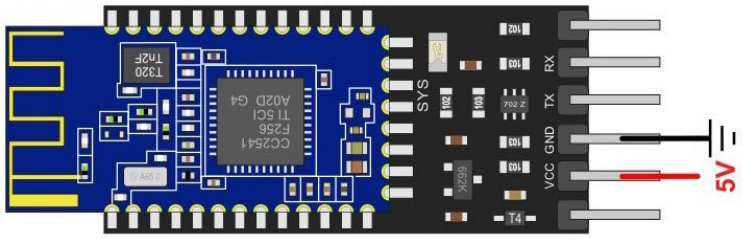


We've already wired up your motor driver and got it working, now let's make it so you can control the Buggy remotely from your phone!

First let me explain how the Bluetooth module works. The actual module itself looks something like this:



Quite scary. But luckily for us, they come soldered onto a breakout board which routes all these ports into a much simpler arrangement to work with.



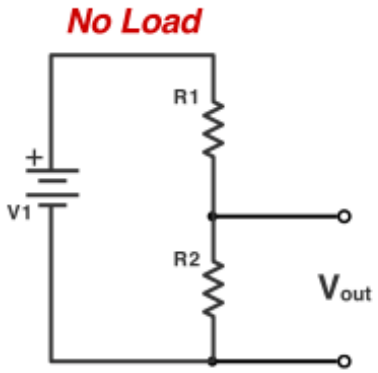
There are 4 pins that we will be working with:

- VCC – Powering the module
- GND
- TX – To send signals (talk)
- RX – To receive signals

The wiring for this is pretty similar to what we've done before, but with a slight catch. The HM-10 is a **3.3V logic** component, which means if we run 5V into it we risk damaging the chip. The **Arduino logic pins usually operates at 5V,** so we need to manipulate the Arduino output to supply our Bluetooth module a lower voltage.

We can do this using a **potential divider**.

A **potential divider (voltage divider)** is a simple circuit that uses **two resistors to reduce a voltage**. It does so by using the following calculation:

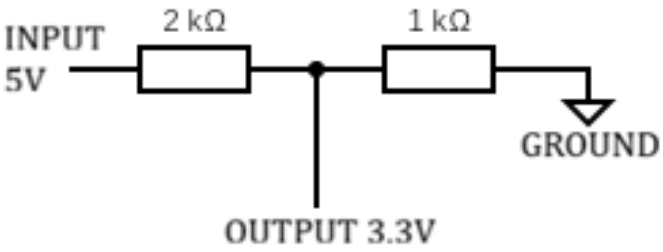


$$V_{out} = V_1 \frac{IR_2}{I(R_1 + R_2)} = \frac{V_1 R_2}{(R_1 + R_2)}$$

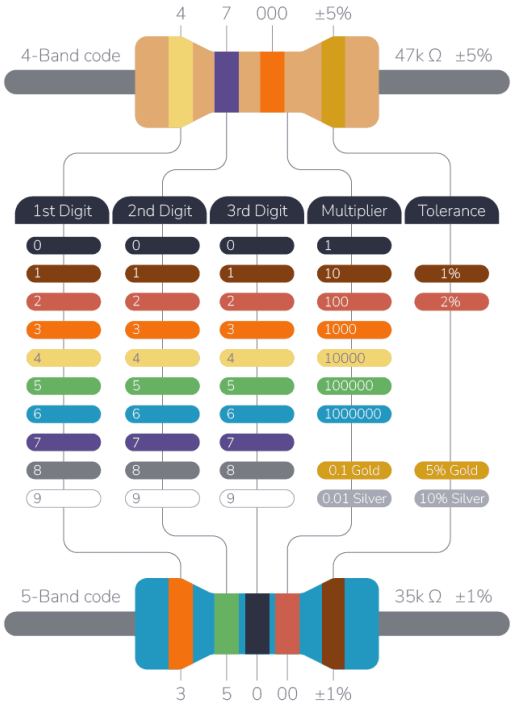
It’s basically like a voltage ratio-er. If you had a 5k resistor and a 3k resistor, an 8V voltage input would be split 5V and 3V across the two resistors respectively.

If you want to know more about how it works, feel free to ask the Buggy team and we’d be happy to explain it to you.

Using the equations displayed above though, we can calculate that to reduce the 5V signal to 3.3V, we need a 2:1 resistor ratio. Then, we arrange it in the following set up:



I recommend using 2k ohm and 1k ohm resistor, but any values work. To figure out the value of your resistor, you can use a resistor value calculator online or use a resistor colour band table as shown below.



ENSURE COMMON GROUND BETWEEN ARDUINO AND HM-10. Make sure they're both connected to the same GND pin on your Arduino Microcontroller. You can do this by utilising the breadboard channels you should have created.

Buggy FAQ: Common Issues & Fixes

Arduino & Code

Code will not upload

- Make sure the Serial Monitor is closed, as it can occupy the serial port.
- Disconnect the Bluetooth module before uploading, as it can interfere with the serial connection.
- Unplug and reconnect the Arduino and try uploading again.
- Check that the correct Arduino board and COM port are selected.
- Save the file and try uploading it again.
- If the problem continues, try another laptop or Arduino.

Arduino only works when connected to a laptop

- Power supply issue.
- Check that the batteries are charged and connected correctly.
- Check the switch and all power and ground connections.
- Check that the Arduino and other components receive sufficient power.
- If the problem continues, check the battery and power wiring.

Code uploads but the buggy behaves incorrectly

- Check that the pin numbers in the code match the physical wiring.
- Check that the correct pins are assigned to the motors, servo, and ultrasonic sensor.
- Check that all wires are connected to the correct pins on the motor driver and Arduino.

Serial Monitor is not showing the expected values

- Make sure Serial.begin() is included in the code.
- Check that the baud rate in the Serial Monitor matches the value used in the code.
- Check that the correct sensor pins are being used.
- Remember to close the Serial Monitor before uploading new code.

Motors & Motor Driver

Motors are not working

- Check the wiring between the Arduino, motor driver, and motors.
- Make sure the motor driver is receiving sufficient power.
- Check that the motors are connected to the correct output terminals.
- Check that the correct Arduino pins are being used in the code.
- Try swapping the motor connections to determine whether the problem is with the motor, wiring, or Arduino pin.
- Some motors may only work reliably at higher speeds, so testing the motor at a speed of 255 can help.

Only one motor/wheel is working

- Check the wiring and motor-driver connections.
- Try switching the motor connections to see whether the problem follows the motor or the connection.
- Check that the relevant Arduino pins are working correctly.

One wheel is moving faster than the other

- This can be caused by differences between motors, wheels, or mechanical friction.
- Adjust the PWM/speed value for each motor independently so that both wheels move at a similar speed.
- Check that both motors are receiving the correct power.
- Check that neither wheel is rubbing against the chassis.

Buggy does not drive straight

- Adjust the speed of each motor independently.
- Check for loose wheels or the sizes of the wheels
- Check that nothing is causing additional friction on one side.
- Different surfaces can also affect how straight the buggy travels.

Motor is spinning in the wrong direction

- Check the IN1/IN2 or IN3/IN4 connections and the corresponding code.
- Swap the motor wires if necessary.
- Alternatively, reverse the HIGH/LOW configuration in the code.

Motor speed control is not working

- Make sure ENA and ENB are connected to PWM-capable pins.
- Check that the ENA and ENB jumpers are set correctly for the chosen wiring.
- Check that the correct speed values are being used in analogWrite().

- If using the Servo library on an Arduino Uno, avoid using pins 9 and 10 for motor PWM.

Motor driver is not working correctly

- Make sure all wires are securely connected to the motor driver.
- Check the connections between the battery, Arduino, and motor driver.
- Check the motor-driver screw terminals for loose wires.
- Try another motor or motor driver, if available, to see which component is causing the problem.

Servo Motor

Servo is not working

- Check the servo's power, ground, and signal connections.
- Make sure the breadboard connections are correct.
- Check that the servo is receiving sufficient power.
- Check that the correct signal pin is being used in the code.

Bluetooth

- Switch your Bluetooth off when not in use as to not interfere with other buggies
- If it’s not connecting properly, could be a power issue. Either replace batteries or add another battery holder

Wiring, Power & Assembly

Buggy is not powering on

- Check that the batteries are inserted in the correct orientation and have sufficient charge.
- Check the switch and its wiring.
- Check all power and ground connections.
- Check that wires are securely connected to the motor driver and breadboard.
- Check that positive and ground wires have not accidentally been connected together.

Lose or broken wires

- Check all soldered connections.
- Check the motor-driver screw terminals for loose wires.
- Make sure wires are firmly secured.

Battery pack is coming loose

- Secure the battery pack properly to the chassis.
- Use tape if needed so they don’t fall out

Components are becoming loose or damaged

- Avoid overtightening chassis bolts, as this can crack plastic components.
- Check and tighten bolts if motors or wheels become loose.
- Make sure components attached to hot glue are securely fixed.

Ultrasonic Sensor (Sem 2)

Ultrasonic sensor gives unreliable or inconsistent readings

- Check the sensor's wiring and power supply.
- Check that the TRIG and ECHO pins in the code match the physical wiring.
- Make sure the sensor is positioned correctly and is not obstructed.
- Test the sensor against a flat object at a known distance.
- If the sensor continues to give unusual readings, it may need replacing